Demo Q&A — Detailed Answers

Expanded, detailed answers to the six questions the assessment's demo is required to cover. Every specific below (bug names, line numbers, thresholds, file names) is verified against the actual code and against `04-trade-offs.md`, not summarized from memory.

Quick-reference summary

#	Question	One-line answer
1	What the project does	Given only a feature + model version, the agent autonomously investigates a drift alert, calls its own tools, writes a 5-section report, and pauses for human approval.
2	AI tools / coding agents used	Claude Code (build), LangGraph Studio (visual debugging), Groq / Anthropic / Hugging Face (the agent's own LLM backends).
3	How the agent helped plan/implement/debug/refactor	4 real architecture rewrites, live bug fixes across all 3 providers, 2 live deployment-bug fixes.
4	Features added / bugs fixed	7-layer guardrail stack, real human-in-the-loop, 3-provider support, Streamlit UI, 20 real bugs fixed.
5	What was cut or simplified	3 other agent ideas cut for missing infra; lineage simplified to a JSON snapshot; root cause stays a labeled hypothesis.
6	Weakest part	`export_drift_summary()` never run against a live Deepchecks `SuiteResult` — shape is documented, not verified.

---

1. What the project does

Aspect	Detail
Base project	A real telecom churn-prediction ML pipeline (`Code+Folder/`) with genuine Deepchecks-based drift detection already implemented in `src/ml_pipeline/drift.py`.

Drift checks it builds on	`WholeDatasetDrift`, `TrainTestFeatureDrift`, `FeatureLabelCorrelationChange` (PPS), `TrainTestLabelDrift` — real thresholds: 0.2 feature/whole-dataset drift, 0.4 label drift, recall < 0.80 / f1 < 0.5 model drift.
What triggers an investigation	Only two inputs: `feature` (e.g. `total_charges`) and `model_version` (e.g. `v14`) — nothing else is handed to the agent.
What the agent decides for itself	Which of 3 tools to call, in what order, and when it has enough evidence — not a fixed script.
The 3 tools	`get_drift_metrics` (weekly PSI/accuracy time series), `get_lineage` (pipeline version history, deployments, incidents), `get_model_metadata` (accuracy/AUC/feature importance over time).
Output	A structured 5-section report: Alert Summary, Root Cause, Statistical Variance, Lineage Context, Recommended Actions.
Final gate	Execution genuinely pauses (LangGraph `interrupt()`) until a human approves or rejects — not a UI illusion.
How it's demoed	CLI (`langgraph_investigator.py`), LangGraph Studio (visual graph + human-in-the-loop UI), and a deployed Streamlit app.

---

2. What AI tools or coding agents I used

Tool	Role
Claude Code	Primary coding agent for the entire build — architecture design, implementation, live debugging, and deployment troubleshooting, across the whole multi-day session.
LangGraph Studio	Visual debugger for the agent's own graph (nodes, edges, state, human-approval interrupt) — required a separate isolated Python 3.11+ venv (`.venv_studio`) since `langgraph-cli[inmem]` needs a newer Python than the main agent env.
Groq (Llama-3.3-70B)	LLM backend #1 for the agent itself — fast, free-tier friendly, but hit real rate limits (100K TPD) during heavy testing.
Anthropic (Claude)	LLM backend #2 — used when Groq was rate-limited; writes noticeably more thorough reports (needed higher `max_tokens`).

Hugging Face (Llama-3.3-70B, free Inference Providers tier)	LLM backend #3, added specifically at my request for a third fallback with zero cost — confirmed tool-calling actually works on the free tier before wiring it in, since not every HF-hosted model supports `bind_tools()` reliably.
`resolve_provider()` / `_get_chat_model()`	The abstraction that lets all of the above sit behind one interface — auto-detects from whichever API key is set (Groq → Anthropic → Hugging Face), or an explicit override.

---

3. How the agent helped me plan, implement, debug, or refactor

Architecture iterations (not one design, four)

Stage	What it was	Why it changed
1. Single-shot LLM call	Python code deterministically loaded the drift report, flagged failing checks, correlated the changelog, then stuffed it all into one prompt.	This is a **workflow**, not an agent — the model never decided anything. Caught and named explicitly as an overclaim.
2. Raw tool-calling loop	`drift_investigator.py` — model given only a `job_id` and 3 tools, decides what to call itself.	First genuinely agentic version, but no visual debugging, no guardrails yet.
3. Fixed-fanout LangGraph	`planner → [drift, lineage, metadata in parallel] → reasoning` — real graph, but tool calls were still hard-coded, not model-decided.	Wanted a *unified* agent with real LLM-driven routing, not a fixed fan-out.
4. Unified agentic graph (final)	`check_drift → agent ↔ {drift, lineage, metadata} → finalize → verify → fact_check → human_approval` — the model itself chooses which of the 3 named tool nodes to call, in what order, looping until done.	This is the actual deliverable — matches the requested "3 nodes: drift, metadata, lineage" shape with genuine routing.

Real bugs caught and fixed live, by provider

Provider	Bug	Root cause
Groq	Crash on zero-arg tool calls	`tc.function.arguments` returns the literal string `"null"`; `json.loads("null")` is Python `None`, not `{}`, crashing `**args` unpacking.

Groq	`tool_use_failed` errors	Llama-3.3 occasionally emits malformed tool-call JSON — transient model noise, fixed with a 3-attempt retry.
Groq / any	Chronologically impossible root cause	The model cited a changelog entry dated *after* the drift because it sounded thematically closer — fixed with an explicit "check the date" instruction in the system prompt.
Anthropic	Guardrail regex crash	`langchain_anthropic`'s `.content` can be a list of content blocks instead of a plain string, even with no tool calls — crashed `verify_narrative_node`. Fixed with a `_message_text()` normalizer.
Anthropic	Truncated reports	No `max_tokens` was set at all (LangChain default ~1024); Claude's reports run longer than Llama's. Raised to 4096.
Anthropic (Studio only)	`thinking.thinking: Field required`	Extended-thinking blocks didn't survive Studio's state serialization/replay. Fixed by stripping them before replay.

Real deployment bugs fixed live

Bug	Fix
Streamlit Cloud: `Invalid format: please enter valid TOML`	Smart-quote auto-substitution corrupting pasted secrets — fixed by retyping directly.
Streamlit Cloud: `ModuleNotFoundError: yaml`	Streamlit Cloud always installs from the **root** `requirements.txt` regardless of UI config — the root file was still the churn-model's heavy pinned deps. Renamed it to `requirements-churn-model.txt` and wrote a new lightweight root file for the agent's actual dependencies.

A refactor driven by a design request, not a bug

Before	After	Why
Generic `ToolNode` dispatching all 3 tools	3 separate named nodes (`drift_tool_node`, `lineage_tool_node`, `metadata_tool_node`), each filtering to only its own tool calls	Explicitly requested: a unified agent with 3 visually distinct nodes in LangGraph Studio, not one opaque "tools" box.

4. What features I added or bugs I fixed

Features added

Feature	Detail
7-layer guardrail stack	Input validation, drift-negative short-circuit, turn cap (6 turns max), 3 deterministic anti-fabrication checks (numbers/versions/incident IDs), structural completeness check, LLM fact-checker, prompt-injection defense in the system prompt.
Real human-in-the-loop	`interrupt()` + `Command(resume=...)`, backed by a `MemorySaver` checkpointer — a genuine pause, not a spinner standing in for one.
Multi-provider support	Groq / Anthropic / Hugging Face, auto-detected or explicitly chosen.
Streamlit UI	Sidebar dropdowns (feature/model_version/audience, all populated dynamically from fixture data), provider selector, approve/reject buttons, tool-call trace viewer, downloadable report.
Expanded drift dataset	3 selectable features: `total_charges` (dramatic, clear cause), `monthly_charges` (stable, no drift), `tenure` (drifting, genuinely no recorded cause) — deliberately built to test honest agent behavior on the "no answer" case, not just the easy one.
Live deployment	Streamlit Community Cloud, debugged through 2 real deploy-only failures.

Bugs fixed — count by category

Category	Count	Examples
Agent-loop / provider bugs	6	Groq null-args crash, Groq malformed tool calls, chronological reasoning, Anthropic content-list crash, token truncation, thinking-block serialization
Guardrail bugs (found by stress-testing the guardrails)	5	Recontextualization gap (motivated `fact_check_node`), case-sensitivity in version extraction, incident-ID regex miss, plus 2 hallucinations the guardrails correctly caught (arithmetic error, timeline miscalculation)

LangGraph Studio / infra	6	Stale graph cache (404), `TypedDict` import bug breaking the input form, `KeyError: 'flagged'` from manual state edits, `KeyError: 'feature'` from missing input, Groq rate limits, Anthropic low balance
Deployment	2	TOML paste corruption, wrong `requirements.txt` being installed
Security	1 (ongoing practice)	3 real API keys pasted into chat across the session — each flagged for rotation immediately, never echoed back, and redacted (63 instances, verified zero leaked) when exporting session logs

Notable hallucinations the guardrails caught, not created bugs of my own: Claude wrote "a -0.06 AUC swing (0.902 → 0.861)" — both numbers were real, but 0.902 − 0.861 = 0.041, not 0.06. The deterministic check flagged `0.06` as absent from tool output; the independent fact-checker separately named the exact arithmetic error and stated the correct value. That's two guardrail layers catching the same real error two different ways.

---

5. What I cut or simplified to keep it focused

Item	Decision	Why
Compliance/audit reporting agent	Cut at the idea stage	Needs a real model registry / audit infrastructure that doesn't exist in this project and couldn't be faked convincingly in the time available.
Smart retraining-cost optimization agent	Cut at the idea stage	Needs historical run-cost data that doesn't exist here either.
GitOps/CI PR reviewer agent	Cut at the idea stage	Buildable, but generic — wouldn't extend this specific codebase the way a drift investigator does.
Real MLflow / model-registry lineage	Simplified to a hand-authored JSON snapshot (`lineage.json`)	This pipeline doesn't use MLflow today — models are saved via `.save_model()`/pickle with no run tracking. The tool is named honestly around this (`get_lineage`, not `get_mlflow_run`) so the demo doesn't imply live integration that isn't there.

Causal inference	Root cause stays explicitly labeled `(Hypothesis)`	The narrative is a correlation-based hypothesis (time-window + feature-name matching), not verified causation — stated plainly rather than overclaimed.
Retry/caching robustness	Mostly single LLM call per run	Only Groq's malformed-tool-call failure mode got a retry (3 attempts) — general robustness wasn't built out further given the 2-day scope.
`export_drift_summary()` live verification	Written against the documented Deepchecks API, not run against a live pinned `deepchecks==0.12.0` stack	Wrapped in try/except so a version mismatch degrades gracefully instead of crashing — see the weakest-part answer below.

---

6. The weakest part, and what I'd improve next

Aspect	Detail
What it is	`export_drift_summary()` in `Code+Folder/src/ml_pipeline/drift.py` — extracts a per-feature drift summary from a Deepchecks `SuiteResult`.
Why it's the weakest part	Written from documented Deepchecks API knowledge (`get_not_passed_checks()` / `get_not_ran_checks()`, the same public API the rest of `drift.py` already relies on), but never executed against a real, live `SuiteResult` in this environment — so its exact output shape for `TrainTestFeatureDrift` / `FeatureLabelCorrelationChange` values is unverified.
Why it doesn't silently fail	Wrapped in try/except, so a version mismatch degrades gracefully rather than crashing the pipeline — but "doesn't crash" isn't the same as "verified correct."
What I'd do next	Run it end-to-end against a real pinned `deepchecks==0.12.0` execution and assert the output schema explicitly, instead of trusting it on documented behavior alone.
Why this is the honest answer, not a deflection	It's a real, verified-live gap I chose to document rather than either hide or falsely claim as fixed — consistent with how the guardrail-testing gaps (recontextualization, case-sensitivity) were handled elsewhere in this build: found, named, and either fixed or explicitly left open with a stated reason.

Source material for all specifics above: `04-trade-offs.md` (bug-by-bug writeups), `langgraph_investigator.py` (verified against the live file, not memory), and this session's Claude Code transcript in `agent_logs/`.